

1. Тестирование ПО и его цели

Тестирование ПО — это процесс проверки соответствия между реальным и ожидаемым поведением программы, выполняемый на конечном наборе тестов.

Основные цели:

- Повышение качества продукта.
- Проверка соответствия требованиям заказчика.
- Обнаружение дефектов на ранних этапах.
- Снижение рисков сбоев на производстве.
- Предоставление информации о состоянии продукта.

2. Тестирование vs Отладка

Различие заключается в целях, исполнителях и процессах:

Критерий	Тестирование	Отладка (Debugging)
Цель	Найти дефекты и подтвердить работоспособность.	Локализовать, понять причину и исправить баг.
Исполнитель	Тестировщик (QA) или автоматизатор.	Разработчик (Developer).
Процесс	Динамический запуск тестов или статический анализ.	Пошаговое выполнение кода, чтение логов, правка.

3. Модульное тестирование (Unit Testing)

Модульное тестирование — проверка минимальных изолированных элементов кода (функций, методов, классов) в отрыве от остальной системы.

- Принцип изоляции: тестируемый объект полностью отделяется от внешних зависимостей (баз данных, API, сетевых запросов).
- Заглушки (Stubs/Mocks):
 - *Stubs* возвращают заранее заготовленные данные на вызовы.
 - *Mocks* проверяют сам факт и параметры вызова (поведение).
- Критерии эффективности: высокая скорость выполнения, независимость тестов друг от друга, стабильность результатов, высокое покрытие кода (Coverage).

4. Тестирование производительности

Проверка скорости, масштабируемости и стабильности системы под нагрузкой.

Виды:

- Нагрузочное (Load): проверка работы при ожидаемой (штатной) нагрузке.
- Стресс-тестирование (Stress): проверка поведения при экстремальных нагрузках и поиск точки отказа.
- Стабильности (Stability/Endurance): проверка долгой работы под средней нагрузкой для выявления утечек памяти.

Метрики отклика:

- Время отклика (Response Time) — задержка между запросом и ответом.
- Пропускная способность (Throughput) — количество запросов в секунду (RPS/TPS).
- Уровень ошибок (Error Rate) — процент неуспешных ответов от общего числа.

5. Пирамида тестирования

Пирамида тестирования — концепция идеального распределения автоматизированных тестов по уровням.

- Unit (Модульные): основа пирамиды (70–80%). Быстрые, запускаются локально.
- Integration (Интеграционные): средний слой (15–20%). Проверяют связку компонентов.
- UI/End-to-End (Пользовательские): вершина (5%). Имитируют действия юзера, медленные и дорогие в поддержке.
- Обоснование: чем ниже уровень теста, тем быстрее он выполняется и тем дешевле обходится локализация и исправление ошибки.

6. Регрессионное тестирование

Регрессионное тестирование — проверка уже протестированных функций после изменения кода (багфикса, рефакторинга, добавления фич) для уверенности, что старая функциональность не сломалась.

- Цель: гарантировать отсутствие новых багов в неизменных частях системы.

- Роль автоматизации: критическая, так как регрессионные тесты запускаются многократно при каждом изменении (в CI/CD pipelines). Вручную выполнять весь объем проверок неэффективно.

7. Тестирование «черного ящика» (Black Box)

Метод тестирования функциональности без доступа к внутренней структуре и коду программы.

- Эквивалентное разделение: разбиение входных данных на группы (классы), где программа ведет себя одинаково. Тестируется одно любое значение из класса.
- Анализ граничных значений (BVA): проверка поведения системы на границах классов эквивалентности, а также на один шаг выше и ниже границ. Пример для диапазона [1...10]: тесты для точек 0, 1, 2, 9, 10, 11.

8. Методология TDD (Test Driven Development)

Разработка, управляемая тестами, когда код пишется строго после написания теста на него.

- Цикл «Красный — Зеленый — Рефакторинг»:
 1. *Красный*: пишется тест на новую функцию. Он падает (Red).
 2. *Зеленый*: пишется минимальный код, чтобы тест прошел (Green).
 3. *Рефакторинг*: код улучшается, устраняется дублирование без изменения поведения.
- Влияние на дизайн: заставляет проектировать код модульным, слабосвязанным и легко тестируемым (иначе на него сложно написать тест заранее).

9. Интеграционное тестирование

Проверка взаимодействия между отдельными программными модулями, подсистемами или сторонними сервисами.

- Снизу вверх (Bottom-up): сборка и тестирование начинаются с низкоуровневых модулей. Для верхних уровней используются эмуляторы вызовов (драйверы).
- Сверху вниз (Top-down): первыми тестируются высокоуровневые управляющие модули. Для замещения нереализованных нижних уровней применяются заглушки (stubs).

10. Тестирование API

Проверка программных интерфейсов без использования графического UI.

Основные проверки:

- Статус-коды: соответствие ответов спецификации (200 OK, 400 Bad Request, 401 Unauthorized, 500 Server Error).
- Структура JSON/XML: валидация схемы ответа, проверка наличия обязательных полей и типов данных.
- Заголовки (Headers): валидация Content-Type, Authorization, кэширования и куки.

11. Приемочное тестирование (Acceptance Testing)

Проверка соответствия продукта бизнес-требованиям для принятия решения о выпуске.

Виды:

- UAT (User Acceptance Testing): тестирование конечными пользователями или заказчиком в реальных сценариях.
- Alpha: внутреннее тестирование сотрудниками компании-разработчика на ранних этапах.
- Beta: внешнее тестирование ограниченной группой реальных пользователей перед релизом.
- Критерии готовности: отсутствие блокирующих багов, успешное выполнение всех критических тест-кейсов, подписание критериев приемки (Acceptance Criteria).

12. Тестирование «белого ящика» (White Box)

Метод тестирования, основанный на анализе внутренней структуры, логики и кода программы.

Метрики покрытия:

- Покрытие операторов (Statement Coverage): процент выполненных строк кода.
- Покрытие ветвей (Branch Coverage): проверка всех исходов условных операторов (например, веток if и else).
- Покрытие условий (Condition Coverage): проверка всех под-условий внутри сложных логических выражений.

13. Жизненный цикл дефекта (Bug Lifecycle)

Путь бага от обнаружения до полной утилизации:

[New] -> [Opened / Assigned] -> [Fixed] -> [Ready for QA] -> [Verified] -> [Closed]

- New: баг зарегистрирован тестировщиком.
- Opened / Assigned: баг изучен тимлидом/PM и назначен на разработчика.
- Fixed: разработчик исправил код.
- Ready for QA: код развернут на тестовом стенде для проверки.
- Verified: тестировщик подтвердил исправление. Если баг воспроизводится, статус меняется на *Reopened*.
- Closed: дефект успешно устранен и закрыт.

14. Статический анализ кода

Анализ кода без его фактического запуска.

- Отличие от динамического: динамическое тестирование проверяет программу во время выполнения; статический анализ ищет потенциальные ошибки, уязвимости и нарушения стандартов в самом тексте кода.
- Линтеры: автоматические инструменты (например, ESLint, Pylint), проверяющие синтаксис, форматирование и типичные ошибки «на лету».
- Код-ревью (Code Review): ручной разбор и оценка кода коллегами-разработчиками для обмена опытом и повышения качества архитектуры.

15. Тестирование мобильных приложений

Обладает спецификой из-за физических ограничений мобильных платформ.

- Фрагментация устройств: необходимость проверять приложение на сотнях комбинаций экранов, процессоров, версий ОС (iOS/Android) и оболочек.
- Управление памятью: проверка поведения при нехватке ОЗУ, сворачивании приложения и очистке фоновых процессов ОС.
- Прерывания (Interrupts): тестирование реакции приложения на входящие звонки, SMS, push-уведомления, потерю сети или разрядку батареи.

16. «Хрупкие» тестов (Brittle Tests)

Тесты, которые часто падают из-за незначительных изменений в системе, не связанных с реальными багами.

- Причины возникновения: использование жестких локаторов в UI (например, полных путей XPath), сильная зависимость от точного порядка данных в БД, привязка к текущему времени или жесткие тайм-ауты (sleep-вызовы).
- Повышение устойчивости: внедрение паттерна *Page Object*, использование уникальных тестовых атрибутов (`data-testid`), динамические ожидания (Explicit Waits) вместо жестких пауз, изоляция тестовых данных.

17. Дымовое (Smoke) и Санитарное (Sanity) тестирование

Два вида быстрых проверок стабильности сборки.

- Дымовое (Smoke): проверка работоспособности самых критичных функций приложения после новой сборки. Отвечает на вопрос: «Работает ли система в целом? Можно ли её тестировать дальше?».
- Санитарное (Sanity): узконаправленная проверка конкретной измененной функции или исправленного бага. Отвечает на вопрос: «Исправлен ли конкретный компонент и не сломал ли он смежную логику?».
- Место в CI/CD: Smoke-тесты запускаются автоматически сразу после деплоя новой сборки в конвейере сборки, блокируя дефектный код до начала масштабного тестирования.

18. Мутационное тестирование

Мутационное тестирование — метод оценки качества самих тестов путем умышленного внесения мелких ошибок (мутаций) в исходный код программы.

- Принцип: специальный инструмент меняет операторы в коде (например, `>` на `<`, или `+` на `-`).
- Убийство мутантов: если после изменения кода хотя бы один тест упал, мутант считается *убитым* (тесты качественные). Если все тесты прошли успешно, мутант *выжил* (тесты плохие, они не замечают подвоха).
- Цель: найти пробелы в тестовом покрытии и логике проверок.

19. Тестирование безопасности

Проверка устойчивости системы к преднамеренным атакам и защита конфиденциальности данных.

- Уязвимость — брешь в системе, позволяющая нарушителю нанести ей вред или обойти правила.
- Аутентификация vs Авторизация:

- *Аутентификация*: проверка подлинности пользователя (Кто ты? Пример: ввод пароля).
- *Авторизация*: проверка прав доступа (Что тебе дозволено? Пример: проверка роли администратора).
- Основные угрозы:
 - *SQL-инъекция*: внедрение вредоносного SQL-кода в поля ввода для несанкционированного доступа к базе данных.
 - *XSS (Cross-Site Scripting)*: внедрение вредоносного скрипта в веб-страницу, который выполняется в браузере других пользователей.

20. Метрики качества ПО

Количественные показатели для оценки состояния продукта и эффективности процессов тестирования.

- Плотность дефектов (Defect Density): количество багов, деленное на размер модуля (на 1000 строк кода или на количество требований).
- Покрытие требований (Requirements Coverage): процент требований, которые проверены хотя бы одним тест-кейсом.
- Эффективность тест-кейсов (Test Case Efficiency): процент тестов, которые находят реальные дефекты, от общего числа запущенных тестов.

21. Исследовательское тестирование (Exploratory Testing)

Одновременное изучение программного продукта, проектирование тестов и их немедленное выполнение.

- Отличие от сценариев: в сценарном тестировании сначала пишут тест-кейсы, а потом строго по ним идут. В исследовательском — тест-дизайн происходит на лету, опираясь на результаты предыдущего шага.
- Роль интуиции: критически важна. Опыт, эвристики и интуиция инженера определяют, в каких именно неочевидных местах системы могут скрываться дефекты. При этом подход системный: тест-сессия ограничивается рамками и целями (чартерами).

22. Верификация vs Валидация в V-модели

Два ключевых процесса проверки качества на зеркальных этапах разработки.

- Верификация (Verification): проверка процесса разработки на соответствие заданным требованиям. Отвечает на вопрос: «*Правильно ли мы создаем продукт?*» (Проверка документации, архитектуры, ревью кода).
- Валидация (Validation): оценка соответствия готового продукта ожиданиям и реальным потребностям пользователей. Отвечает на вопрос: «*Правильный ли продукт мы создали?*» (Запуск программы, проверка сценариев использования).

23. Тестирование локализации (L10n)

Адаптация программного продукта под особенности конкретной страны, языка и культуры.

Проверяются:

- Форматы дат и времени: *дд/мм/гггг* vs *мм/дд/гггг*, 12-часовой vs 24-часовой формат.
- Валюты и числа: разделители разрядов (запятая или пробел), корректность знаков валют (\$, €, ₹) и их положение (до или после числа).
- Unicode-символы: поддержка диакритических знаков, иероглифов, специфических алфавитов и правильное отображение строк без их обрезки (слома верстки).

24. Матрица прослеживаемости требований (Traceability Matrix)

Таблица, связывающая бизнес-требования или техническое задание (ТЗ) с созданными тест-кейсами.

- Цель: наглядно показать, все ли требования покрыты тестами, и быстро определить, какие именно тесты нужно переписать, если требование в ТЗ изменится.

25. Тестирование баз данных

Проверка корректности хранения, извлечения и модификации данных бэкендом.

- Целостность: проверка ограничений (Constraints: *NOT NULL*, *UNIQUE*, *FOREIGN KEY*) — база не должна принимать некорректные данные.
- ACID: проверка транзакций на Атомарность, Согласованность, Изолированность и Долговечность.

- Триггеры и хранимые процедуры: тестирование внутренней логики БД как отдельного программного кода путем передачи граничных и невалидных значений.

26. Конфигурационное тестирование

Проверка работоспособности программного обеспечения на различном аппаратном и системном окружении.

Проверяется стабильность работы при:

- Различных ОС (Windows 10/11, macOS, дистрибутивы Linux) и их версиях.
- Разном железе (объем ОЗУ, типы процессоров x86/ARM, видеокарты).
- Различных версиях сопутствующего ПО (драйверы, браузеры, библиотеки).

27. Тестирование удобства использования (Usability)

Оценка того, насколько интерфейс программы понятен и комфортен для конечного пользователя.

Ключевые критерии:

- Обучаемость (Learnability): насколько легко пользователю выполнить простую задачу при первом знакомстве с интерфейсом.
- Запоминаемость (Memorability): легко ли вспомнить, как работать с программой после долгого перерыва.
- Удовлетворенность (Satisfaction): вызывает ли визуальное и логическое оформление системы приятные эмоции, нет ли раздражающих факторов.

28. Техники тест-дизайна

Методы проектирования оптимального набора тестов.

- Таблицы принятия решений (Decision Tables): структурированная таблица, где в строках указываются комбинации входных условий, а на пересечениях — ожидаемые действия системы. Применяется для сложной бизнес-логики со множеством ветвлений.
- Тестирование переходов состояний (State Transition): моделирование поведения системы, которая меняет свои состояния (например, заказ на

маркетплейсе: *Новый* -> *Оплачен* -> *Доставлен* -> *Закрыт*) в ответ на события. Тесты проверяют корректность всех переходов и невозможность запрещенных.

29. Тестовые артефакты

Документы, создаваемые и используемые в процессе тестирования.

- Test Case (Тест-кейс): атомарный сценарий проверки, содержащий предусловия, шаги воспроизведения и ожидаемый результат.
- Test Suite (Тест-сьют): логическая группа или набор тест-кейсов, объединенных общей целью (например, набор тестов для авторизации).
- Test Plan (Тест-план): стратегический документ, описывающий цели, объем работ, график, ресурсы, риски и критерии оценки всего процесса тестирования на проекте.

30. Тестирование документации

Статический метод проверки требований (ТЗ, макетов, спецификаций) до начала написания кода.

- Полнота: проверка, описаны ли все возможные сценарии поведения системы (включая обработку ошибок).
- Непротиворечивость: контроль отсутствия логических конфликтов (когда в начале ТЗ кнопка описана как синяя, а в конце — как зеленая).
- Цель: исправить ошибку на этапе текста, что в сотни раз дешевле, чем исправлять её в готовом коде.

31. Роль фикстур (Fixtures в Pytest)

Фикстуры — функции в тестовых фреймворках (например, Pytest), которые подготавливают внешнюю среду перед запуском тестов и очищают ее после.

- Setup: код до ключевого слова `yield`, подготавливает состояние (например, создает подключение к БД).
- Teardown: код после `yield`, убирает за собой (например, закрывает соединение, удаляет временные файлы).
- Области видимости (Scope) в Pytest:
 - `function` (по умолчанию): запускается для каждого теста отдельно.
 - `class`: один раз на класс тестов.
 - `module`: один раз на файл с тестами.
 - `session`: один раз за весь цикл прогона тестов.

32. Тестирование установки (Installation Testing)

Проверка корректности развертывания и удаления программного продукта на устройстве пользователя.

- Инсталляция: проверка установки «с нуля», нехватки места на диске, прерывания процесса установки, выбора кастомных путей.
- Обновление (Upgrade): проверка наката новой версии поверх старой с сохранением пользовательских данных и настроек.
- Удаление (Uninstall): проверка полного удаления программы, очистки реестра и удаления всех созданных файлов.

33. Исследовательское vs Ad-hoc тестирование

Два подхода к тестированию без жестких тест-кейсов, которые часто путают.

Критерий	Исследовательское (Exploratory)	Ad-hoc («на коленке»)
Системность	Высокая. Есть четкая цель (чартер), временные рамки (тайм-боксы) и фокус на область.	Низкая. Хаотичные клики по экрану без выраженной методологии.
Документирование	Фиксируются заметки о поведении, найденные пути и аномалии.	Результат фиксируется только в случае нахождения бага (заводится баг-репорт).

34. Инструменты для модульного тестирования

Фреймворки, предоставляющие синтаксис для написания юнит-тестов и автоматические утилиты для их запуска (runners).

- JUnit: стандартный фреймворк для языка Java. Использует аннотации `@Test`, `@BeforeEach`, `@AfterEach`.
- NUnit: популярный фреймворк для платформы .NET (C#), развившийся из JUnit. Использует атрибуты `[Test]`, `[SetUp]`.
- MSTest / Встроенные средства Visual Studio: родной инструмент Microsoft для .NET. Интегрирован прямо в среду разработки, позволяя запускать тесты через окно "Test Explorer" и проводить анализ покрытия кода одной кнопкой.

35. Системы баг-трекинга

Инструменты для регистрации, приоритизации и контроля жизненного цикла задач и дефектов.

- Jira: международный промышленный стандарт. Мощная, кастомизируемая система с поддержкой Agile-досок (Scrum/Kanban), гибкой настройкой бизнес-процессов (Workflows) и сложными фильтрами поиска (JQL).
- YouTrack: баг-трекер от компании JetBrains. Отличается высокой скоростью работы, продвинутым текстовым поиском с автодополнением и удобной клавиатурной навигацией для разработчиков.

36. Этапы процесса сопровождения (поддержки) ПО

Сопровождение — этап жизненного цикла ПО, начинающийся после передачи продукта в эксплуатацию.

Основные этапы:

1. Прием обращения: фиксация заявки или баг-репорта от пользователя.
2. Анализ: верификация проблемы, локализация ошибки в программном модуле, оценка влияния на систему.
3. Проектирование и разработка: создание плана изменений, написание или исправление кода.
4. Тестирование (Регрессионное): проверка исправленного модуля и системы в целом.
5. Внедрение: деплой обновления на рабочую среду (Production).

37. Типы сопровождения

Согласно стандартам инженерии ПО, поддержка делится на четыре вида:

- Исправительное (Corrective): реактивное устранение обнаруженных пользователями багов и отказов.
- Адаптивное (Adaptive): изменение ПО для его корректной работы в изменившихся внешних условиях (например, адаптация под новую версию ОС или СУБД).
- Совершенствующее (Perfective): доработка, улучшение производительности или добавление новых функций по запросу пользователей.
- Профилактическое (Preventive): поиск и устранение скрытых проблем, рефакторинг, улучшение читаемости кода для предотвращения багов в будущем.

38. Организация работы с заявками пользователей

Процесс выстраивается через методологии ITIL/ITSM и специализированные Service Desk системы.

1. Многоуровневая поддержка (L1/L2/L3):
 - *L1 (Первая линия)*: колл-центр/чат. Принимают заявки, решают типовые проблемы по инструкциям.
 - *L2 (Вторая линия)*: системные администраторы, инженеры поддержки. Разбираются в логах, конфигурируют систему.
 - *L3 (Третья линия)*: разработчики и QA. Исправляют баги в коде модуля.
 2. SLA (Service Level Agreement): жесткий регламент, определяющий время реакции на заявку и время её решения в зависимости от критичности (Severity).
-

39. Контроль версий (Git, SVN) в поддержке кода

Системы контроля версий (VCS) хранят историю изменений кода и позволяют вести параллельную работу.

Роль в поддержке:

- Изоляция изменений: исправление багов для поддержки ведется в отдельных ветках (`hotfix` / `bugfix`), не мешая разработке новых крупных фич в ветке `develop`.
- Откат изменений (Rollback): если обновление вызвало критический сбой, VCS позволяет мгновенно вернуть код модуля к предыдущему стабильному состоянию.
- История (Blame/Log): возможность посмотреть, кто, когда и зачем изменил конкретную строку кода, вызвавшую ошибку.

40. Понятие рефакторинга и его значение для поддержки

Рефакторинг — процесс изменения внутренней структуры кода без изменения его внешнего поведения (код делает то же самое, но написан чище).

Значение для поддержки:

- Снижает технический долг (Technical Debt).
- Повышает читаемость кода, делая его понятным для новых инженеров поддержки.
- Облегчает внесение изменений (снижает риск того, что правка в одном месте сломает логику в другом).

41. Характеристики качества ПО по ISO/IEC 25010

Стандарт выделяет 8 ключевых характеристик верхнего уровня:

1. Функциональная пригодность: выполняет ли ПО задачи пользователя должным образом.
2. Эффективность производительности: скорость работы, использование ресурсов (ОЗУ, CPU).
3. Совместимость: способность взаимодействовать с другим ПО.
4. Удобство использования (Usability): простота и понятность интерфейса.
5. Надежность: устойчивость к отказам, способность восстанавливаться после сбоев.
6. Защищенность (Security): безопасность данных и защита от атак.
7. Сопровождаемость (Maintainability): легкость анализа, тестирования и изменения кода модулей.
8. Мобильность (Portability): легкость переноса ПО в другое окружение.

42. Тестовое покрытие (Code Coverage) и как его измерить

Code Coverage — метрика, показывающая, какой процент исходного кода программы был выполнен во время запуска тестов.

- Как измерить: используются специализированные инструменты (например, *JaCoCo* для Java, *Coverage.py* для Python, *Istanbul* для JS). Они внедряют маркеры в код (инструментируют его) перед запуском, а после прогона тестов формируют отчет с точным указанием невыполненных строк, условий или ветвей.

43. Как определить критерии начала и окончания тестирования

Регламентированные условия фаз QA-процесса (Entry/Exit Criteria).

- Критерии начала (Entry Criteria):
 - Готовность тестовой среды (стенда).
 - Требования/ТЗ заморожены и согласованы.
 - Код модуля написан, успешно собран и прошел Smoke-тест.
- Критерии окончания (Exit Criteria):
 - Все запланированные тест-кейсы выполнены.
 - Достигнут целевой уровень тестового покрытия (например, >80%).
 - Нет открытых критических багов (блокирующих и критических приоритетов).
 - Время или бюджет, отведенные на итерацию тестирования, исчерпаны (с оценкой оставшихся рисков).

44. Роль стандартизации в тестировании программных модулей

Стандарты (например, ISO/IEC/IEEE 29119 для процессов тестирования, IEEE 829 для тестовой документации) играют ключевую роль в индустрии.

- Единый язык: обеспечивают общую терминологию, исключая недопонимание между разработчиками, QA и заказчиками по всему миру.
- Стандартизация процессов: предоставляют проверенные лучшие практики (Best Practices) построения QA-отделов.
- Предсказуемость результатов: гарантируют, что независимо от смены команды, качество тестирования модулей останется на высоком, измеримом уровне.